

Machine Learning I Project Proposal

Diabetes Risk Prediction

Silvia Andreeva(9434320258), Arthur Belloc(9038420268),
Artemis Maigne(9019220266), Josep Monotoro(9446020255)

Proposal

Create and evaluate a machine learning model that would be able to predict the diabetes risk using health and lifestyle indicators.

Motivation and Background on Diabetes

Diabetes is a condition that is common among people of different stages of life. There are around 537 million diagnosed people worldwide, and experts predict that the number will continue to rise in the future. It is a condition that occurs when your blood sugar level is too high. It develops when the person's pancreas isn't making enough insulin(or any) or when the person's body isn't properly responding to insulin's effects. For glucose, which is the most important source of energy for the body, to reach the final destination, it needs insulin, and if the body isn't using the hormone correctly, the glucose builds up in the bloodstream, which causes high blood sugar(hyperglycemia). High blood sugar over a prolonged time could cause other health problems such as cardiovascular issues, nerve damage, and eye issues. Diabetes, in most cases, is a chronic disease, but it can be managed with medications and/or changes to one's lifestyle[1].

Objectives

Build binary classification models that can predict if a person has or doesn't have diabetes. Create, evaluate, and compare different machine learning models to find the best model based on our goals. Assess the health impact of misclassification, as false negatives will delay the treatment of a patient, while false positives could cause unnecessary stress, both mentally and financially. This impact will be incorporated into model selection. Strive to identify the key risk factors that contribute to diabetes.

Dataset

The dataset chosen is the [CDC Diabetes Health Indicators Dataset](#), which contains both healthcare statistics and lifestyle information about people and their diagnosis of diabetes. There are 253,000 instances with 21 features, including BMI, High Cholesterol, Smoker, Income, Age, and Sex. Most variables are binary variables, with a few being integer, and there are no missing values.

Implementation

1. *Data cleaning and preprocessing*

Data cleaning and preprocessing are mandatory for this project, as they will make sure that the dataset is used to the fullest. The dataset needs to be cleaned first in order to ensure that any NULL or unwanted parameters do not alter the results of our learning models.

The dataset chosen was already labeled as precleaned. However, to validate that the following steps were taken:

- Check the target distribution: The dataset is clearly unbalanced, with only 14% of diabetes results. This information is important because, due to this result, applying class weight to the models could help achieve better overall accuracy.

- Check the duplicates: around 9% of the dataset are duplicates. This is real people's data, hence the possibility of two people having the same results is quite high. Hence, we decided to not remove the duplicates.
- Check missing values: There were no missing values, confirming that the dataset was already precleaned.

Overall, the dataset is a good fit for the objectives, as it is relevant, with a lot of data, and is clean with no missing values. The only characteristic of the dataset that could pose problems in the training of classification models is unbalanced classes. Therefore, a scaling could be useful in order to yield better results.

2. *Preprocessing*

The dataset was split using a stratified split with a random state of 42 and a test size of 20%.

The random state was used for the reproducibility of the stratification. The code for this can be found in the appendix.

3. *Logistic Regression*

Logistic Regression will serve as the base model, as it is the simplest model to implement for classification. It will produce a binary result (Diabetes or non Diabetes). This model has its limits due to its low complexity, but it could still produce interesting results if tuned properly. Moreover, those results will be used as a threshold of quality. If a more complex model yields the same or worse accuracy, it means that the model is either not good for our data or not tuned well. Along with that, scaling of the training set will be done because the dataset is heavily unbalanced, and scaling may lead to better results. Therefore, three different logistic regression models have been trained.

- Normal Logistic Regression:

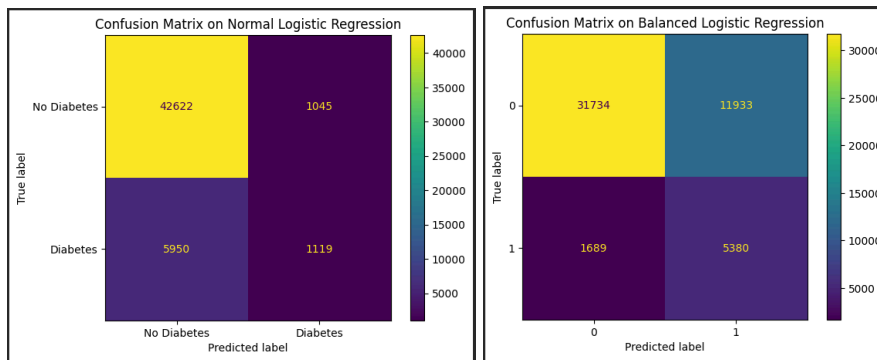
Classification Report:				
	precision	recall	f1-score	support
0	0.88	0.98	0.92	43667
1	0.52	0.16	0.24	7069
accuracy			0.86	50736
macro avg	0.70	0.57	0.58	50736
weighted avg	0.83	0.86	0.83	50736

This is the baseline for the project. Here, it can be observed that the accuracy is 86%, which is good. However, flaws can be noted. The objective is for the model to be able to classify correctly as many true positives as possible, while minimizing the amount of false negatives. The classification report shows that the recall for class 1 is 0.16, which means that only 16% of the true positives were accurately predicted. The reason for this is most likely the class imbalance. Therefore, the next logistic regression model will have balanced weights in order to maximize the recall on the minority class.

- Balanced Logistic Regression:

Classification Report:				
	precision	recall	f1-score	support
0	0.95	0.73	0.82	43667
1	0.31	0.76	0.44	7069
accuracy			0.73	50736
macro avg	0.63	0.74	0.63	50736
weighted avg	0.86	0.73	0.77	50736

The report shows that the recall has significantly improved to 76%, but the overall accuracy has decreased to 73%. The false negatives have become significantly less, while the true positives have increased. To compare, the confusion matrices for the two models have been generated:



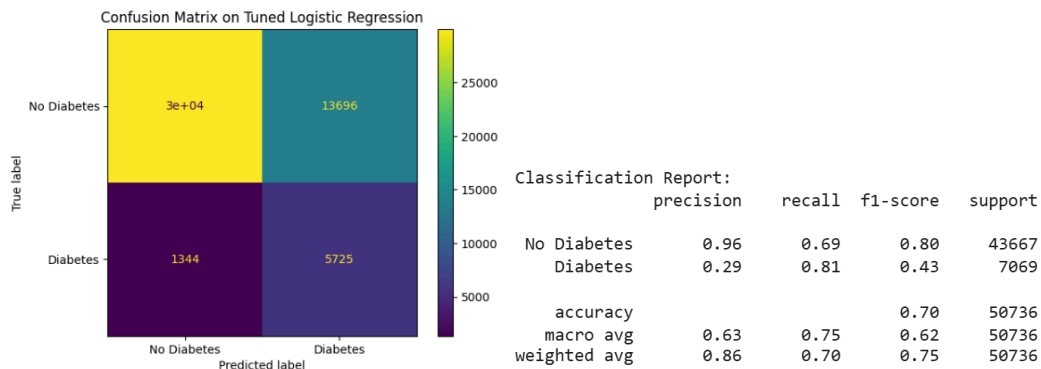
Normal Logistic Regression

Balanced Logistic Regression

The last step taken to ensure the best logistic regression model for this dataset has been found is to fine-tune the model. Multiple models were trained in a loop by changing the hyperparameters: C, penalty, solver, class_weight, and threshold.

- **Tuned Logistic Regression:**

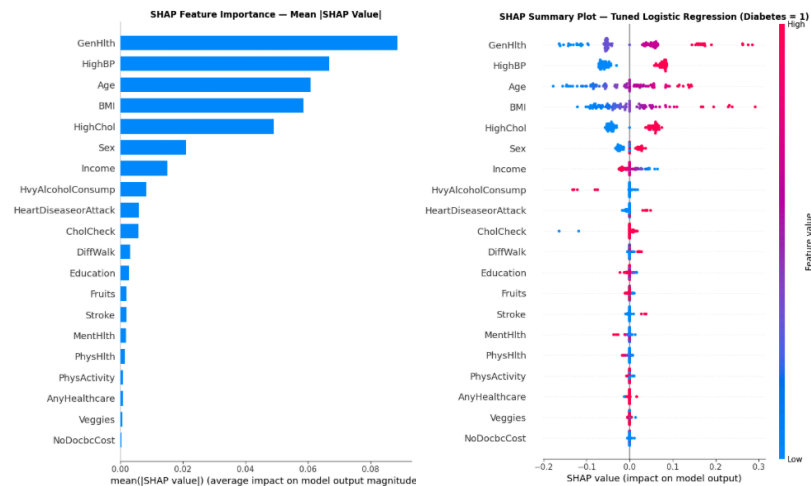
The aim is to find a better model than the balanced logistic regression model. The best model found after tuning the hyperparameters has the following metrics:



The model uses the L2 penalty, balanced class weight, and liblinear solver. It has strong regularization of 0.001, and the classification threshold was 0.46. On the validation dataset, it produced an overall 0.70 accuracy, and it predicted true positives correctly in 81% of the cases. On the test data, it also produced 0.81 recall on the diabetes class. While the overall accuracy has decreased compared to the previous models, the aim of this project is not to get the best accuracy, but to minimize the false negatives while not compromising too much the overall accuracy.

SHAP Analysis:

The most important variables for the prediction were GenHlth, HighBP, Age, BMI, and HighChol, while Veggies, PhysActivity, and NoDocbcCost were the least significant. If a patient has a higher GenHlth score(poor health), has high blood pressure, is older, and/or has high BMI, they would be more likely to have diabetes according to the model.



4. Random Forest

The next model type implemented was Random Forest, which is a good fit because it is able to capture nonlinear relationships and it is robust to overfitting. Additionally, Random Forest is able to handle a large number of features and measures feature importance. The initial model implemented was with 1100 estimators and parallel processing was enabled for faster computation. The final prediction was obtained through a majority vote among all trees in the forest. Once trained, the model was evaluated on the test dataset. The classification report shows good overall accuracy, but its recall on the diabetes class is very low, which signifies that it misses a lot of the true positives, which is not the objective.

	precision	recall	f1-score	support
0	0.87	0.99	0.93	43667
1	0.58	0.13	0.21	7069
accuracy			0.87	50736
macro avg	0.73	0.56	0.57	50736
weighted avg	0.83	0.87	0.83	50736

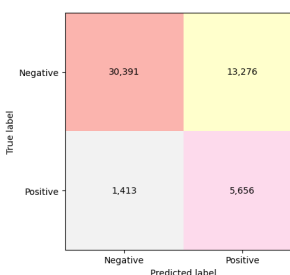
Finetuning

To improve the performance of the Random Forest classifier, hyperparameter tuning was performed using RandomSearchCV. Ten randomly selected hyperparameters combinations were tested using 3-fold cross-validation. The search space included the following parameters: number of estimators, maximum tree depth, minimum samples required for splitting, minimum samples per leaf, and class weight. The best-performing combination was selected and used to train the final model. Along with that, threshold tuning was done to achieve the best recall. A 5-fold cross-validation on the training data was used to generate out-of-fold predictions. The best threshold was selected to achieve at least 80% recall while still maximizing the precision.

Results and best model:

Best parameters: {'n_estimators': 400, 'min_samples_split': 2, 'min_samples_leaf': 10, 'max_depth': 10, 'class_weight': 'balanced'}

	precision	recall	f1-score	support
0	0.96	0.70	0.81	43667
1	0.30	0.80	0.44	7069
accuracy			0.71	50736
macro avg	0.63	0.75	0.62	50736
weighted avg	0.86	0.71	0.75	50736



The best threshold to achieve desired recall was 0.4764. The recall was 0.80, while the overall accuracy was 71%. These adjustments improved the model's ability to correctly identify diabetic patients and reduced the number of false negatives, which is particularly critical in a medical context.

Feature Importance:

The fitted Random Forest model provides the feature importance scores, and they show the relative contribution of each attribute to the model's prediction. The most important features are GenHlth, HighBP, BMI, HighChol, and Age.

	Feature	Importance
0	GenHlth	0.239562
1	HighBP	0.219550
2	BMI	0.135792
3	HighChol	0.098441
4	Age	0.096564
5	DiffWalk	0.057195
6	HeartDiseaseorAttack	0.028551
7	Income	0.028367
8	PhysHlth	0.027915
9	Education	0.011783

5. *Gradient Boosting*

The third type of model explored was Gradient Boosting. LightGBM was chosen as it uses tree-based learning algorithms, and it is capable of handling large amounts of data. It is faster for training, highly efficient, and in general, leads to better accuracy. It also supports GPU.

Three different models were evaluated:

- Gradient Boosting with a custom loss function:

Because the data is highly imbalanced and the goal is to minimize the false negatives, a custom loss function that is asymmetric and cost-sensitive was implemented. For a true positive, the model is rewarded with + 2, while for a false positive, it is punished with -2, and for a false negative with -10. False negatives were penalized five times more heavily than false positives. This made the model focus more on spotting the true positives. The loss function is based on the weighted binary cross-entropy gradient and Hessian derivation [2], modified to incorporate a cost matrix. The positive class weight is defined as $C_{FN} - C_{TP}$, while the negative class weight is C_{FP} . The loss function is:

$$L(y, \hat{y}) = -[(C_{FN} - C_{TP}) * y * \log(\hat{y}) + C_{FP} * (1 - y) * \log(1 - \hat{y})]$$

- Gradient Boosting(LightGBM with Binary Logistic Objective):

This model's objective will be the standard logistic regression - 'binary'.

- Balanced Gradient Boosting(LightGBM with Binary Logistic Objective):

This model's objective will be the standard logistic regression - 'binary'. Along with that, the scale_pos_weight will be equal to the class difference ratio of 6.2. This is done because the dataset is imbalanced.

Finetuning

For finetuning, the method used was a randomized grid search with cross-validation. The parameters on which fine-tuning was performed were the number of estimators, the learning rate, the number of leaves, the minimum child samples, the feature fraction, the bagging fraction, lambda L1, and lambda L2. Along with that, SMOTE was used to generate synthetic data to balance the target classes, but that yielded worse results, so no further experimentation was done with this function and model. The cross-validation was done with 5 folds. Along with that, as a custom objective was used, the LightGBM model returns raw scores instead of predicted probabilities. Hence, a custom scorer (raw_score_roc_auc(estimator, X,y)) was created that evaluates AUC directly from the raw scores during the cross-validation.

Lastly, to determine the best threshold that maximizes the recall without compromising the overall accuracy, a 5-fold cross-validation on the training data was used to generate out-of-fold predictions. A range of thresholds was evaluated, and the resulting precision and recall scores were calculated. The best threshold was selected so that at least 80% recall can be achieved, which was the main objective of this study, while still maximizing the precision. After obtaining both the best model and best threshold, the model predicted the raw scores for the test set, and the AUC and classification report were computed.

Results and best model:

Model #	Objective	Accuracy	Recall	Precision	TP	TN	AUC
Model 1	Custom loss	0.71	0.81	0.96	5,739	30,098	0.8273
Model 2	Binary	0.71	0.81	0.96	5,761	30,130	0.82789
Model 3	Binary + scale_pos_weight	0.71	0.81	0.96	5,759	30,063	0.8275

The test AUC score was around 0.827 for all models, which shows that they are able to discriminate well between diabetic and non-diabetic patients. All three models have almost the same classification report. As can be observed, for class 0, the models have high precision(0.96) and good recall(0.69), which means that when the models predict non-diabetic, they are almost always correct, but they miss 30% of the true non-diabetic people. For class 1, the precision is relatively low(0.3), but the recall is good. This means that the models can predict correctly 81% of the diabetic patients, but only 30% of the predicted diabetics are actually diabetic. This is consistent with the goal, which is to accurately diagnose everyone with diabetes and minimize false negatives.

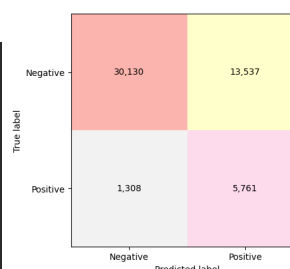
Interestingly, all models have very similar results, even though the first model incorporates false-negative and false-positive costs into the loss function, and the third model uses scale_pos_weight to account for data imbalance. Neither of these functions improved the model's discriminative ability. Overall, in the end, the simplest Gradient Boosting model resulted in the best recall and accuracy for the dataset.

Best Model - Gradient Boosting(LightGBM with Binary Logistic Objective):

Best parameters: {'num_leaves': 15, 'n_estimators': 500, 'min_child_samples': 20, 'learning_rate': 0.05, 'lambda_l2': 1, 'lambda_l1': 1, 'feature_fraction': 0.6, 'bagging_fraction': 0.8}

Best threshold: -1.922232462804065

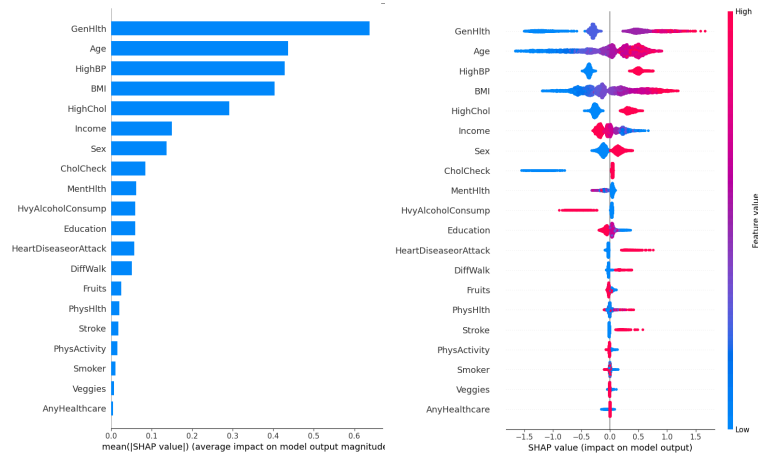
Classification Report					
	precision	recall	f1-score	support	
0	0.96	0.69	0.80	43667	
1	0.30	0.81	0.44	7069	
accuracy			0.71	50736	
macro avg	0.63	0.75	0.62	50736	
weighted avg	0.87	0.71	0.75	50736	



Interpretability of the Gradient Boosting Model 2

Using SHAP, the most important features and their impact on the prediction can be identified. The graph on the left shows how important each feature is and which are the strongest predictors. As it can be observed, the general health score, high blood pressure, age, BMI, and high cholesterol are the most significant attributes. Physical activity, being a smoker, and eating veggies seem to have a negligible effect on the predictions. The graph on the right shows how each feature affects the prediction, with the right side indicating a higher chance of

predicting diabetic, while the left side indicates non-diabetic. A high value of general health (poor health), high blood pressure, older age, higher BMI, high cholesterol, and being a man increase the risk of being diabetic.



6. Neural Network(MLP)

This section presents the development, tuning, and evaluation of a Multilayer Perceptron (MLP) neural network for binary diabetes classification. The same preprocessed dataset from Part 1 was used. The primary challenge addressed is the 6:1 class imbalance between non-diabetic and diabetic samples, which requires careful threshold calibration to maximize clinically relevant Recall.

Model Development — Iterative Approach

Four versions of the MLP were developed, each building on findings from the previous:

- MLP Original: GridSearchCV (120 fits, ROC-AUC scoring) with default threshold 0.50. Established a strong ROC-AUC (0.826), but very low Recall (0.135) due to class imbalance.
- MLP v2: Same architecture, threshold tuned to maximize F1-Score. Recall improved from 13.5% to 63.1% with no change in ROC-AUC. Selected as the final model.
- MLP v3: SMOTE oversampling + F1 scoring (~7 hrs training). ROC-AUC dropped to 0.772 — synthetic samples did not generalize. Discarded.
- MLP v4: Deeper architecture search — 8 network sizes, wider alpha range, learning rate tuning (640 fits, ~10 hrs). Marginal ROC-AUC gain (+0.004 vs v2). Not worth the compute cost.

Final Model — MLP v2 Hyperparameters

Parameter	Value
hidden_layer_sizes	(128, 64)
activation	relu
alpha	0.0001
solver	adam
early_stopping	True
max_iter	300
Classification threshold	0.19 (tuned for max F1)

Table 1: MLP v2 final hyperparameters.

Threshold Tuning

The default threshold of 0.50 produced a Recall = 0.135 — missing 6,113 of 7,069 diabetic patients. Since ROC-AUC = 0.826 confirmed the model had strong discriminative ability, the low Recall was a threshold calibration problem, not a learning failure. The optimal threshold (0.19) was found by maximizing F1-Score on the Precision-Recall curve.

Evaluation Results

MLP Version Comparison

Version	Accuracy	Precision	Recall	F1-Score	ROC-AUC
MLP Original (thr=0.50)	0.8652	0.5687	0.1352	0.2185	0.8261
MLP v2 (thr=0.19) ✓	0.7985	0.3693	0.6305	0.4658	0.8261
MLP v3 (SMOTE+F1)	0.7550	0.3083	0.6098	0.4095	0.7723
MLP v4 (Deep Arch)	~0.799	0.3740	0.6220	0.4670	0.8304

Table 2: MLP Comparison

Confusion Matrix — MLP v2 (threshold = 0.19)

	Predicted: 0	Predicted: 1
Actual: 0 (Healthy)	36,055 (TN)	7,612 (FP)
Actual: 1 (Diabetic)	2,612 (FN)	4,457 (TP)

Table 3: Confusion matrix for MLP v2.

Compared to the original model (956 TP), MLP v2 correctly identified 4,457 diabetic patients — a 366% improvement in true positive detection. The increase in false positives (725 7,612) is an acceptable trade-off: in medical screening, missing a positive case carries far greater clinical cost than a false alarm that leads to a follow-up test.

SHAP Explainability Analysis

MLP has a black-box nature, and because of that, it is less interpretable than tree-based models at the neuron level. SHAP can be used to partially address this limitation as it quantifies each feature's contribution to individual predictions. SHAP KernelExplainer was applied to MLP v2 (100 background samples, 100 test observations). Sampling was used to reduce the computational cost, and because of this, the following results can be considered an approximation of the results obtained when using the whole dataset.

Top Features by Mean |SHAP| Value

Rank	Feature	Mean SHAP	Direction
1	GenHlth (General Health)	0.042	High value (poor health) → risk ↑
2	BMI	0.039	Higher BMI → risk ↑
3	Age	0.027	Older age → risk ↑
4	HighBP	0.025	Presence → risk ↑
5	HighChol	0.021	Presence → risk ↑
6	Income	0.010	Lower income → risk ↑
7	HeartDiseaseorAttack	0.008	Presence → risk ↑
8	Sex	0.007	Mixed direction

Table 4: Top 8 features ranked by mean absolute SHAP value.

Key SHAP findings

GenHlth and BMI dominate the model's decisions by a large margin. The beeswarm plot confirms that both show monotonic positive relationships — higher feature values consistently push predictions toward diabetes. Dependence plots for Age show a clear threshold effect: scaled values below 0 are protective, while older ages cluster near neutral or slightly positive SHAP contributions. Features such as Smoker, Veggies, Fruits, and NoDocbcCost showed minimal SHAP impact across all test samples.

Notably, the top 5 SHAP features identified by the MLP (GenHlth, BMI, Age, HighBP, HighChol) are **consistent with the Gradient Boosting SHAP analysis** from Part 3, reinforcing the robustness of these findings across different model families.

Best neural network model

MLP v2 — a two-layer network (128, 64) with ReLU activation and a classification threshold of 0.19 — was selected as the final neural network model. Threshold tuning improved Recall from 13.5% to 63.1%, detecting 3,501 additional diabetic patients while maintaining ROC-AUC = 0.826. SHAP analysis confirmed that General Health, BMI, and Age are the dominant predictors, consistent with findings from the Gradient Boosting model and established clinical literature.

7. **Cross-Model Comparison**

The table below consolidates results from all models. Recall is the primary metric given the clinical goal of minimizing missed diabetes diagnoses.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.7036	0.2948	0.8099	0.4322	0.8197
Random Forest	0.71	0.30	0.80	0.44	0.822
Gradient Boosting	0.71	0.30	0.81	0.44	0.82789
MLP v2	0.7985	0.3693	0.6305	0.4658	0.8261

Table 5: Cross-model comparison. Green cells = best per metric.

Gradient Boosting and Logistic Regression models achieved the highest Recall of 0.81, indicating that these models accurately predict 81% of positive cases. They also maintain an overall accuracy of 71% and an AUC score of around 0.827, which shows good discriminability. Gradient boosting and neural network models are theoretically capable of capturing more complex nonlinear relationships than logistic regression. However, for this implementation, they did not substantially outperform logistic regression. One possible explanation for that is that logistic regression underwent a more extensive hyperparameter search. Its computational cost is relatively low, so around six thousand combinations were able to be tested. On the other hand, the computational time for training gradient boosting and neural network models is far longer, and the search space is far wider. Hence, far fewer hyperparameter combinations were explored, and, consequently, the reported performance of these models may not represent their fully optimized performance. The second possible explanation is that the limitation comes from the information contained in the features. As can be observed in the comparison table, all the models had very similar AUC scores. ROC-AUC score signifies how well the model is able to discriminate between the two classes. The similarity across models shows that even simpler models work quite well, and complex ones do not improve the performance much. The flexibility and ability to capture non-linear relationships do not lead to significantly better discrimination between diabetic and non-diabetic individuals.

In terms of interpretability, all models' SHAP analysis shows the same top-5 features: GenHlth, BMI, Age, HighBP, and HighChol. Hence, we can conclude that these are the most important features that allow the model to discriminate between diabetic and non-diabetic patients.

Conclusion

Overall, since all the models have very similar AUC values and demonstrate strong discriminatory ability between diabetic and non-diabetic patients, the best-performing model was the Gradient Boosting with binary logistic objective. Its AUC is marginally higher than the rest, and it captures the highest number of true positives without significantly reducing the overall accuracy, which is the main objective of this project.

Peer Evaluation:

Silvia Andreeva - 33

Arthur Belloc - 31

Artemis Maigne - 5

Josep Monotoro - 31

References

[1] Cleveland Clinic, "Diabetes: What It Is, Causes, Symptoms, Treatment & Types," Cleveland Clinic, [Online]. Available: <https://my.clevelandclinic.org/health/diseases/7104-diabetes>. [Accessed: May 24, 2026].

[2] C. Wang, C. Deng, and S. Wang, "Imbalance-XGBoost: leveraging weighted and focal losses for binary label-imbalanced classification with XGBoost," *arXiv:1908.01672*, Aug. 2021.

Appendix

Libraries:

```
!pip install ucimlrepo
from ucimlrepo import fetch_ucirepo
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import warnings
warnings.filterwarnings('ignore')
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, StratifiedKFold, RandomizedSearchCV, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix,
    classification_report, ConfusionMatrixDisplay, RocCurveDisplay, make_scorer, roc_curve,
    precision_recall_curve)
from sklearn.neural_network import MLPClassifier
from sklearn.utils.class_weight import compute_sample_weight
import lightgbm as lgb
from lightgbm import LGBMClassifier
from imblearn.over_sampling import SMOTE
import shap
```

Code for cleaning and preprocessing:

```
cdc_diabetes_health_indicators = fetch_ucirepo(id=891)
X = cdc_diabetes_health_indicators.data.features
y = cdc_diabetes_health_indicators.data.targets
print("X shape:", X.shape)
print("y shape:", y.shape)
target_col = y.columns[0]
df = pd.concat([X, y], axis=1)
display(df.head())
df.info()
# unique values for each column
for col in df.columns:
    print("\n" + "="*50)
    print(col)
    print(df[col].value_counts().sort_index())
display(df.describe().T) # basic statistics

# Target Distribution
print("Target value counts:")
print(df[target_col].value_counts())
print("\nTarget proportions:")
print(df[target_col].value_counts(normalize=True) * 100)

# Checking the duplicates
duplicate_count = df.duplicated().sum()
print("Number of duplicated rows:", duplicate_count)
print("Percentage of duplicated rows:", duplicate_count / len(df) * 100)

# Checking the missing values
print("\nTotal missing values:", df.isnull().sum().sum())

# Separating features and target
X_clean = df.drop(columns=[target_col])
y_clean = df[target_col]
```

```

print("X_clean shape:", X_clean.shape)
print("y_clean shape:", y_clean.shape)

# Train test split
X_train, X_test, y_train, y_test = train_test_split(X_clean, y_clean, test_size=0.2,
                                                    random_state=42, stratify=y_clean)

```

Code for Logistic Regression:

```

# Scaling the data to yield better results
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# normal logistic regression
log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_scaled, y_train)
y_pred = log_reg.predict(X_test_scaled)
y_proba = log_reg.predict_proba(X_test_scaled)[: , 1]
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)

print("ROC-AUC:", roc_auc)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
ConfusionMatrixDisplay.from_estimator(log_reg, X_test_scaled, y_test,
                                     display_labels=["No Diabetes", "Diabetes"])
plt.title("Confusion Matrix on Normal Logistic Regression")
plt.show()

# Logistic regression with balanced weights
log_reg_balanced = LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42)
log_reg_balanced.fit(X_train_scaled, y_train)
y_pred_balanced = log_reg_balanced.predict(X_test_scaled)
y_proba_balanced = log_reg_balanced.predict_proba(X_test_scaled)[: , 1]
print("ROC-AUC:", roc_auc_score(y_test, y_proba_balanced))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_balanced))

ConfusionMatrixDisplay.from_estimator(log_reg_balanced, X_test_scaled, y_test)
plt.title("Confusion Matrix on Balanced Logistic Regression")
plt.show()

# Tuned Logistic Regression
C_values = [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100]
threshold_values = np.arange(0.05, 0.96, 0.01)
configs = [
    {"penalty": "l2", "solver": "lbfgs", "class_weight": None},
    {"penalty": "l2", "solver": "lbfgs", "class_weight": "balanced"},
    {"penalty": "l1", "solver": "liblinear", "class_weight": None},
    {"penalty": "l1", "solver": "liblinear", "class_weight": "balanced"},
    {"penalty": "l2", "solver": "liblinear", "class_weight": None},
    {"penalty": "l2", "solver": "liblinear", "class_weight": "balanced"}]

results = []
models = {}
for config_id, config in enumerate(configs):
    for C in C_values:
        model = LogisticRegression(

```

```

C=C, penalty=config["penalty"], solver=config["solver"],
class_weight=config["class_weight"], max_iter=3000, random_state=42)
model.fit(X_train_model_scaled, y_train_model)
val_proba = model.predict_proba(X_val_scaled)[: , 1]
for threshold in threshold_values:
    val_pred = (val_proba >= threshold).astype(int)
    model_name = (
        f"LR_penalty={config['penalty']}_"
        f"solver={config['solver']}_"
        f"class_weight={config['class_weight']}_"
        f"C={C}")
    results.append({
        "model_name": model_name,
        "config_id": config_id,
        "penalty": config["penalty"],
        "solver": config["solver"],
        "class_weight": config["class_weight"],
        "C": C,
        "alpha": 1 / C,
        "threshold": threshold,
        "accuracy": accuracy_score(y_val, val_pred),
        "precision": precision_score(y_val, val_pred, zero_division=0),
        "recall": recall_score(y_val, val_pred, zero_division=0),
        "f1_score": f1_score(y_val, val_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_val, val_proba)})
    models[model_name] = model
results_df = pd.DataFrame(results)
display(results_df.head())

MIN_ACCURACY = 0.70
validation_rows = []
for _, row in results_df.iterrows():
    model_name = row["model_name"]
    threshold = row["threshold"]
    model = models[model_name]
    val_proba = model.predict_proba(X_val_scaled)[: , 1]
    val_pred = (val_proba >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_val, val_pred).ravel()
    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    validation_rows.append({
        **row.to_dict(),
        "tn": tn, "fp": fp, "fn": fn, "tp": tp,
        "sensitivity_recall_diabetes": sensitivity,
        "specificity_no_diabetes": specificity,
        "false_negative_rate": fn / (fn + tp) if (fn + tp) > 0 else 0,
        "false_positive_rate": fp / (fp + tn) if (fp + tn) > 0 else 0})
validation_results_df = pd.DataFrame(validation_rows)
valid_medical_models = validation_results_df[
    validation_results_df["accuracy"] >= MIN_ACCURACY].copy()
print("Number of models tested:", len(validation_results_df))
print("Number of models with accuracy >=", MIN_ACCURACY, ":", len(valid_medical_models))
if len(valid_medical_models) == 0:
    raise ValueError(
        "No model reached the minimum validation accuracy. "
        "Try lowering MIN_ACCURACY or expanding the hyperparameter search.")
best_medical_models = valid_medical_models.sort_values(
    by=["recall", "fn", "f1_score", "precision", "roc_auc", "accuracy"],
    ascending=[False, True, False, False, False, False])
display(best_medical_models.head(20))

```

```

best_row = best_medical_models.iloc[0]
best_model_name = best_row["model_name"]
best_threshold = best_row["threshold"]
best_model = models[best_model_name]
print(best_model_name)
print("\nBest threshold:", best_threshold)
print("\nValidation results:")
display(best_row)
test_proba = best_model.predict_proba(X_test_scaled)[: , 1]
test_pred = (test_proba >= best_threshold).astype(int)
print("ROC-AUC:", roc_auc_score(y_test, test_proba))
print("\nClassification Report:")
print(classification_report(
    y_test,
    test_pred,
    target_names=["No Diabetes", "Diabetes"],
    zero_division=0))

ConfusionMatrixDisplay.from_predictions(y_test, test_pred,
    display_labels=["No Diabetes", "Diabetes"])
plt.title("Confusion Matrix on Tuned Logistic Regression")
plt.show()

# SHAP code
background = shap.sample(X_train_model_scaled, 100, random_state=42)
explainer = shap.KernelExplainer(best_model.predict_proba, background)
X_test_sample = X_test_scaled[:100]
shap_values = explainer.shap_values(X_test_sample, nsamples=100)
if isinstance(shap_values, list):
    shap_vals_pos = shap_values[1]
else:
    shap_vals_pos = shap_values[:, :, 1] if shap_values.ndim == 3 else shap_values

expected_val = (
    explainer.expected_value[1]
    if isinstance(explainer.expected_value, (list, np.ndarray))
    else explainer.expected_value)
feature_names = list(X_train.columns)
shap.summary_plot(
    shap_vals_pos, X_test_sample, feature_names=feature_names, show=False)

plt.title("SHAP Summary Plot — Tuned Logistic Regression (Diabetes = 1)", fontweight="bold")
plt.tight_layout()
plt.show()

shap.summary_plot(
    shap_vals_pos, X_test_sample, feature_names=feature_names, plot_type="bar", show=False)

plt.title("SHAP Feature Importance — Mean |SHAP Value|", fontweight="bold")
plt.tight_layout()
plt.show()

```

Code for Random Forest:

```

rf = RandomForestClassifier(n_estimators=1100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
params = {
    'n_estimators': [200, 400, 600],

```

```

    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 5, 10],
    'class_weight': [None, 'balanced']}
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
random_search = RandomizedSearchCV(
    estimator=rf, param_distributions=params,
    n_iter=10, cv=3, scoring='roc_auc',
    n_jobs=-1, verbose=3, random_state=42)
random_search.fit(X_train, y_train)
best_rf = random_search.best_estimator_
best_parameters = random_search.best_params_
print(best_parameters)

# threshold finetuning
y_scores_cv = cross_val_predict(
    best_rf, X_train, y_train, cv=5, method="predict_proba", n_jobs=-1)[: , 1]

precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores_cv)
target_recall = 0.80
valid_idx = np.where(recalls[:-1] >= target_recall)[0]
if len(valid_idx) == 0:
    raise ValueError(f"No threshold achieves recall >= {target_recall}")
best_idx = valid_idx[np.argmax(precisions[:-1][valid_idx])]
chosen_threshold = thresholds[best_idx]
print(f"Chosen threshold: {chosen_threshold}")

# model performance of test data
y_scores_test = best_rf.predict_proba(X_test)[: , 1]
y_pred_test = (y_scores_test >= chosen_threshold).astype(int)
print(classification_report(y_test, y_pred_test))
test_auc = roc_auc_score(y_test, y_scores_test)
print(f"AUC: {test_auc}")
cm = confusion_matrix(y_test, y_pred_test)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Negative', 'Positive'])
disp.plot(cmap="Pastel1_r", colorbar=False)
for i in range(2):
    for j in range(2):
        disp.text_[i, j].set_color('black')
        disp.text_[i, j].set_text(f"{cm[i, j]:,}")
plt.show()

# Feature Importance
feature_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': best_rf.feature_importances_})
feature_importance = feature_importance.sort_values(
    by='Importance', ascending=False).reset_index(drop=True)
print(feature_importance.iloc[:10])

```

Code to create the Gradient Boosting Model:

```

def loss_function(y_real, y_pred):
    false_negative = 10.0
    false_positive = 2.0
    true_positive = 2
    probabilities = 1.0/(1.0 + np.exp(-y_pred))

```

```

    gradient =
probabilities*(false_positive*(1-y_real)+y_real*(false_negative-true_positive))-(false_negative-true_positive)*y
_real
    hessian = probabilities*(1.0-probabilities)*(false_positive*(1.0 -y_real) +
y_real*(false_negative-true_positive))
    hessian = np.maximum(hessian,1e-4)
    return gradient, hessian

def raw_score_roc_auc(estimator, X,y):
    raw_scores = estimator.predict(X, raw_score=True)
    return roc_auc_score(y, raw_scores)

# code run to do finetuning
base_model = LGBMClassifier(
    objective=loss_function, # change to 'binary' for the logistic regression model2 and 3
    random_state=42,verbose=1,device='gpu', n_jobs=-1) #add scale_pos_weight = 6.2 for model 3
parameter_grid = {
    "n_estimators": [300, 500, 1000],
    "learning_rate": [0.005, 0.01, 0.02, 0.05],
    "num_leaves": [15, 31, 63, 127],
    "min_child_samples": [20, 50, 100, 200],
    "feature_fraction": [0.6, 0.8, 1.0],
    "bagging_fraction": [0.6, 0.8, 1.0],
    "lambda_l1": [0, 0.1, 1, 5],
    "lambda_l2": [0, 0.1, 1, 5]}
random_search = RandomizedSearchCV(
    estimator=base_model, param_distributions=parameter_grid, n_iter=15,cv=5,
    scoring=raw_score_roc_auc, random_state=42, verbose=3, n_jobs=1,return_train_score=False)
random_search.fit(X_train, y_train)
best_params = random_search.best_params_
print(best_params)
print(random_search.best_score_)

# Predictions for best threshold
kfolds = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof_scores = np.zeros(len(X_train))
for train_idx, validation_idx in kfolds.split(X_train, y_train):
    X_fold_train = X_train.iloc[train_idx]
    y_fold_train = y_train.iloc[train_idx]
    X_fold_validation = X_train.iloc[validation_idx]
    model = LGBMClassifier(objective=loss_function, # change to 'binary' for model 2 and model 3
        random_state=42, verbose=-1, #add scale_pos_weight = 6.2 for model 3
        device='gpu', n_jobs=-1,**best_params)
    model.fit(X_fold_train, y_fold_train)
    oof_scores[valid_idx] = model.predict(X_fold_validation, raw_score=True)

# testing only some of the thresholds
possible_thresholds = np.percentile(oof_scores, np.linspace(0.1, 99.9, 1000))
best_threshold = None
best_precision = -1
for threshold in possible_thresholds:
    pred = (oof_scores > threshold).astype(int)
    precision = precision_score(y_train, pred)
    recall = recall_score(y_train, pred)
    if recall >= 0.814:
        if precision > best_precision:
            best_precision = precision
            best_threshold = threshold

print(best_threshold)

```

```

best_model = random_search.best_estimator_
# Test the best model with the best threshold
test_scores = best_model.predict(X_test, raw_score=True)
print(roc_auc_score(y_test, test_scores))
y_pred = (test_scores > best_threshold).astype(int)
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Negative', 'Positive'])
disp.plot(cmap="Pastell_r", colorbar=False)
for i in range(2):
    for j in range(2):
        disp.text_[i,j].set_color('black')
        disp.text_[i,j].set_text(f'{cm[i,j]:,}')
plt.show()

# SHAP Analysis
sample_size = min(5000, len(X_test))
X_test_sample = X_test.sample(sample_size, random_state=42)
explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_test_sample)
shap.summary_plot(shap_values, X_test_sample)
shap.summary_plot(shap_values, X_test_sample, plot_type="bar")

```

Code for MLP:

```

# Original MLP
param_grid = {
    'hidden_layer_sizes': [(64,), (128,), (64, 32), (128, 64)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001, 0.01],
    'max_iter': [300],
    'early_stopping': [True],
}
# 4 × 2 × 3 = 24 combinations × 5 folds = 120 fits
import time
mlp = MLPClassifier(random_state=42)
grid = GridSearchCV(mlp, param_grid, cv=5, scoring='roc_auc', n_jobs=-1, verbose=3)
start = time.time()
grid.fit(X_train_scaled, y_train)
elapsed = time.time() - start
m, s = divmod(int(elapsed), 60)
print(f"\nSearch completed in {m:02d}:{s:02d}")
print(f"Best ROC-AUC CV : {grid.best_score_: .4f}")
print(f"Best params : {grid.best_params_}")
best_mlp = grid.best_estimator_
# Evaluate MLP Original
y_pred = best_mlp.predict(X_test_scaled)
y_pred_prob = best_mlp.predict_proba(X_test_scaled)[ :, 1]
acc = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_pred_prob)
f1 = f1_score(y_test, y_pred)

print("MLP Original Evaluation:")
print(f"Accuracy : {acc: .4f}")
print(f"Precision : {precision: .4f}")
print(f"Recall : {recall: .4f} ← very low due to class imbalance")
print(f"F1-Score : {f1: .4f}")
print(f"ROC-AUC : {auc: .4f}")
print(classification_report(y_test, y_pred))

```

```
# MLP Original: Confusion Matrix + ROC Curve
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
# Confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axes[0], colorbar=False)
axes[0].set_title("MLP Original — Confusion Matrix\n(threshold = 0.50)", fontweight="bold")
# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
axes[1].plot(fpr, tpr, lw=2, color="#2e6db4", label=f"MLP Original (AUC = {auc:.3f})")
axes[1].plot([0,1],[0,1], 'k--', lw=1)
axes[1].set_xlabel("False Positive Rate")
axes[1].set_ylabel("True Positive Rate")
axes[1].set_title("MLP Original — ROC Curve", fontweight="bold")
axes[1].legend(loc="lower right")
plt.tight_layout()
plt.savefig("mlp_original_eval.png", dpi=150)
plt.show()
print("→ ROC-AUC = 0.826 shows strong discriminative ability.")
print("→ But Recall = 0.135 means 6,113 diabetic patients were missed.")
print("→ Problem: default threshold 0.50 is too conservative for imbalanced data.")
```

```
# MLP v2 - Threshold Tuning
# Find optimal threshold (maximizes F1)
precisions, recalls, thresholds = precision_recall_curve(y_test, y_pred_prob)
f1_scores = 2 * (precisions[:-1] * recalls[:-1]) / (precisions[:-1] + recalls[:-1] + 1e-9)
best_thresh_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_thresh_idx]

print(f"Optimal threshold : {best_threshold:.3f} (default was 0.50)")
print(f"→ Precision   : {precisions[best_thresh_idx]:.3f}")
print(f"→ Recall      : {recalls[best_thresh_idx]:.3f}")
print(f"→ F1          : {f1_scores[best_thresh_idx]:.3f}")
```

```
# Evaluate MLP v2 with tuned threshold
y_pred_tuned = (y_pred_prob >= best_threshold).astype(int)
acc_v2 = accuracy_score(y_test, y_pred_tuned)
precision_v2 = precision_score(y_test, y_pred_tuned)
recall_v2 = recall_score(y_test, y_pred_tuned)
auc_v2 = roc_auc_score(y_test, y_pred_prob)
f1_v2 = f1_score(y_test, y_pred_tuned)
print(" MLP v2 Evaluation (Tuned Threshold)")
print(f"Accuracy   : {acc_v2:.4f}")
print(f"Precision  : {precision_v2:.4f}")
print(f"Recall     : {recall_v2:.4f} ← +366% vs Original")
print(f"F1-Score   : {f1_v2:.4f} ← +113% vs Original")
print(f"ROC-AUC    : {auc_v2:.4f} ← unchanged (threshold-independent)")
print(classification_report(y_test, y_pred_tuned))
```

```
# MLP v2: Confusion Matrix comparison + ROC Curve
fig, axes = plt.subplots(1, 3, figsize=(18, 4))
# Original confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axes[0], colorbar=False)
axes[0].set_title("MLP Original\n(threshold = 0.50)", fontweight="bold")
# v2 confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_tuned, ax=axes[1], colorbar=False)
axes[1].set_title(f"MLP v2 — Tuned Threshold\n(threshold = {best_threshold:.2f})", fontweight="bold")
# ROC Curve (same model, threshold shown)
fpr, tpr, roc_thresholds = roc_curve(y_test, y_pred_prob)
```

```

axes[2].plot(fpr, tpr, lw=2, color="#2e6db4", label=f"MLP v2 (AUC = {auc_v2:.3f})")
axes[2].plot([0,1],[0,1], 'k--', lw=1)
# Mark the chosen threshold point
idx_thresh = np.argmax(np.abs(roc_thresholds - best_threshold))
axes[2].scatter(fpr[idx_thresh], tpr[idx_thresh], color='red', zorder=5, s=80,
                label=f"Threshold = {best_threshold:.2f}")
axes[2].set_xlabel("False Positive Rate")
axes[2].set_ylabel("True Positive Rate")
axes[2].set_title("ROC Curve with Selected Threshold", fontweight="bold")
axes[2].legend(loc="lower right")
plt.tight_layout()
plt.savefig("mlp_v2_eval.png", dpi=150)
plt.show()
print("→ TP went from 956 → 4,457 (detected 3,501 more diabetic patients)")
print("→ FN went from 6,113 → 2,612 (missed far fewer diabetic patients)")
print("→ ROC-AUC unchanged: 0.826 — model quality was always good")

```

```

# MLP v3 - Smote
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)
# MLP v3 — Hardcoded results from previous run
# Best params: activation=tanh, alpha=0.0001, hidden_layer_sizes=(128, 64), learning_rate_init=0.001
# Optimal threshold: 0.53
acc_v3, precision_v3, recall_v3, f1_v3, auc_v3 = 0.7550, 0.3083, 0.6098, 0.4095, 0.7723
cm_v3 = [[33993, 9674], [ 2758, 4311]]
print("MLP v3 Results (SMOTE + F1, threshold=0.53)")
print(f"Accuracy : {acc_v3:.4f}")
print(f"Precision : {precision_v3:.4f}")
print(f"Recall : {recall_v3:.4f}")
print(f"F1-Score : {f1_v3:.4f}")
print(f"ROC-AUC : {auc_v3:.4f} ← dropped vs v2 (0.8261)")
print("\n→ SMOTE generated synthetic samples that did not generalize well.")
print("→ ROC-AUC dropped from 0.826 → 0.772. Model discarded.")
# MLP v3: Confusion Matrix (from hardcoded values)
cm_v3_arr = np.array([[33993, 9674], [ 2758, 4311]])
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_v3_arr, display_labels=[0, 1])
disp.plot(ax=ax, colorbar=False)
ax.set_title("MLP v3 — SMOTE + F1\n(threshold = 0.53)", fontweight="bold")
plt.tight_layout()
plt.savefig("mlp_v3_confusion.png", dpi=150)
plt.show()
print("→ Despite similar Recall to v2, ROC-AUC is lower → worse underlying model.")

```

```

# Best params found after 640-fit search:
best_params_v4 = dict(hidden_layer_sizes=(128, 64), activation='relu',
                      alpha=0.1, learning_rate_init=0.0005,
                      max_iter=300, early_stopping=True)
best_mlp_v4 = MLPClassifier(solver='adam', random_state=42, **best_params_v4)
best_mlp_v4.fit(X_train_scaled, y_train)

# MLP v4 — Hardcoded results from previous run
# GridSearch Best ROC-AUC CV: 0.8304
# Optimal threshold: 0.219
acc_v4, precision_v4, recall_v4, f1_v4, auc_v4 = 0.7985, 0.374, 0.622, 0.467, 0.8304
cm_v4_arr = np.array([[36681, 6986], [ 2672, 4397]])
print("MLP v4 Results (Deep Arch, threshold=0.22)")

```

```

print(f"Accuracy : {acc_v4:.4f}")
print(f"Precision : {precision_v4:.4f}")
print(f"Recall : {recall_v4:.4f}")
print(f"F1-Score : {f1_v4:.4f}")
print(f"ROC-AUC : {auc_v4:.4f} ← best ROC-AUC overall (+0.004 vs v2)")
print("\n→ v4 has best ROC-AUC but v2 has better Recall (0.6305 vs 0.6220).")
print("→ Marginal improvement after 10 hours. MLP v2 selected as final model.")
# MLP v4: Confusion Matrix (from hardcoded values)
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_v4_arr, display_labels=[0, 1])
disp.plot(ax=ax, colorbar=False)
ax.set_title("MLP v4 — Deep Architecture\n(threshold = 0.22)", fontweight="bold")
plt.tight_layout()
plt.savefig("mlp_v4_confusion.png", dpi=150)
plt.show()

```

```

# SHAP Explainability Analysis
background = shap.sample(X_train_scaled, 100, random_state=42)
explainer = shap.KernelExplainer(best_mlp.predict_proba, background)
X_test_sample = X_test_scaled[:100]
shap_values = explainer.shap_values(X_test_sample, nsamples=100)
print("SHAP values computed")
# Handle both list and 3D array output formats
if isinstance(shap_values, list):
    shap_vals_pos = shap_values[1]
else:
    shap_vals_pos = shap_values[:, :, 1] if shap_values.ndim == 3 else shap_values
expected_val = (explainer.expected_value[1]
                 if isinstance(explainer.expected_value, (list, np.ndarray))
                 else explainer.expected_value)
feature_names = list(X_train.columns)

```

```

# SHAP Summary Plot (Beeswarm)
shap.summary_plot(shap_vals_pos, X_test_sample, feature_names=feature_names, show=False)
plt.title("SHAP Summary Plot — MLP v2 (Diabetes = 1)", fontweight="bold")
plt.tight_layout()
plt.savefig("shap_summary.png", dpi=150, bbox_inches='tight')
plt.show()
print("→ Pink/red dots = high feature value; Blue = low feature value.")
print("→ Points right of center = pushes toward Diabetes prediction.")
# SHAP Feature Importance Bar Plot
shap.summary_plot(shap_vals_pos, X_test_sample, feature_names=feature_names,
                  plot_type="bar", show=False)
plt.title("SHAP Feature Importance — Mean |SHAP Value|", fontweight="bold")
plt.tight_layout()
plt.savefig("shap_bar.png", dpi=150, bbox_inches='tight')
plt.show()
print("→ GenHlth and BMI have the largest average impact on predictions.")
print("→ NoDocbcCost, Smoker, Veggies have minimal impact.")

```

```

# MLP version comparison table
comparison_all = pd.DataFrame({
    "Version": ["MLP Original", "MLP v2 (Threshold) ✓", "MLP v3 (SMOTE + F1)", "MLP v4 (Deep Arch)"],
    "Accuracy": [round(acc,4), round(acc_v2,4), round(acc_v3,4), round(acc_v4,4)],
    "Precision": [round(precision,4), round(precision_v2,4), round(precision_v3,4), round(precision_v4,4)],
    "Recall": [round(recall,4), round(recall_v2,4), round(recall_v3,4), round(recall_v4,4)],
    "F1-Score": [round(f1,4), round(f1_v2,4), round(f1_v3,4), round(f1_v4,4)],
    "ROC-AUC": [round(auc,4), round(auc_v2,4), round(auc_v3,4), round(auc_v4,4)]})
print("MLP Full Evolution Comparison")
print(comparison_all.to_string(index=False))

```

```

# Visual comparison bar chart
metrics = ["Accuracy", "Precision", "Recall", "F1-Score", "ROC-AUC"]
versions = comparison_all["Version"].tolist()
colors_bar = ["#aaaaaa", "#2e6db4", "#e05c5c", "#f0a500"]
x = np.arange(len(metrics))
width = 0.2
fig, ax = plt.subplots(figsize=(12, 5))
for i, (_, row) in enumerate(comparison_all.iterrows()):
    bars = ax.bar(x + i*width, [row[m] for m in metrics],
                  width, label=row["Version"],
                  color=colors_bar[i], edgecolor="white",
                  linewidth=0.6,
                  alpha=1.0 if i == 1 else 0.75) # highlight v2
ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(metrics, fontsize=11)
ax.set_ylim(0, 1.12)
ax.set_ylabel("Score", fontsize=11)
ax.set_title("MLP Version Comparison — All Metrics", fontsize=13, fontweight="bold")
ax.legend(loc="upper right", fontsize=9)
ax.axhline(0.8, color="gray", linestyle=":", lw=0.8, alpha=0.5)
plt.tight_layout()
plt.savefig("mlp_version_comparison.png", dpi=150)
plt.show()
print("→ Blue bars (MLP v2) show the best overall balance of Recall and F1.")
print("→ v4 has slightly higher ROC-AUC but required 10x more training time.")

```